

Where to Use What — Course Cheat Sheet

One line per tool: what it is, and when to reach for it. Sessions in brackets. Mental model for the whole course: **serve the model (S1)** → **package and deploy it (S2)** → **gate its releases (S3)** → **watch it (S4)** → **upgrade it safely (S5)** → **build the product on it (S6–S9)** → **govern it (S10)**.

1 · Model Serving (S1)

Tool	Simple definition	Use it when
FastAPI	Python web framework — you write the serving endpoint yourself.	Demos, small internal tools, baselines; any model type; lowest learning curve.
BentoML	Packages model + code + dependencies into one deployable artifact; serving and Docker image generation built in.	You want a clean, reproducible path from model to container without writing a Dockerfile.
vLLM	High-throughput LLM inference engine (PagedAttention + continuous batching), OpenAI-compatible API.	Self-hosted LLMs with real traffic; concurrent users; tokens/sec and p95 under load matter. Needs Linux + NVIDIA GPU.
SGLang	vLLM-class LLM engine with prefix caching (RadixAttention) and fast structured/JSON output.	Many requests share long system prompts; strict JSON-schema outputs; agent loops replaying context.
NVIDIA Triton	Industrial inference server running many frameworks (PyTorch, ONNX, XGBoost...) on shared GPUs with dynamic batching.	Enterprise fleets serving MANY mixed models with strict SLOs and a platform team to run it.
KServe	Kubernetes-native serving platform — you declare an InferenceService; it runs engines (vLLM/Triton) inside, with autoscaling and scale-to-zero.	Your organisation lives on Kubernetes and many teams deploy many models.

Rule of thumb: Start with the simplest stack that meets today's traffic; benchmark before you believe. Same GPU + higher tokens/sec = cheaper tokens — cost per 1M tokens = hourly instance cost ÷ (tokens/sec × 3600) × 1,000,000.

2 · Packaging & Deployment (S2)

Tool / concept	Simple definition	Use it when
Docker image	A sealed, reproducible snapshot of app + dependencies + (optionally) model weights.	Always — the standard unit of deployment. Pin base images and package versions.
docker compose	Runs a multi-container stack (app + monitoring + proxy) from one YAML file.	Local development and labs; the local stand-in for a cloud runtime.
HEALTHCHECK / probes	Automated 'is the kitchen ready?' checks; traffic is withheld until healthy.	Every model server — model loading takes minutes; never route traffic before readiness.
Secret manager / .env	Secrets injected at runtime; never baked into images or committed to git.	Any credential. Local: .env (+ .gitignore/.dockerignore). Cloud: Secret Manager + IAM.
Cloud Run / App Runner / Container Apps	Managed container runtimes: give them an image, get a URL + autoscaling + scale-to-zero.	Default cloud target for HTTP model APIs without a platform team.
Managed Kubernetes (GKE/AKS/EKS)	Provider runs the control plane; you run workloads with full orchestration power.	Shared GPU fleets, many teams/models, custom networking. Self-managed K8s only for on-prem / data-residency constraints.

3 · Release Safety — CI/CD, Canary, Rollback (S3, S5)

Tool / concept	Simple definition	Use it when
Promptfoo	Declarative eval runner: YAML of test questions + assertions against your endpoint; non-zero exit on failure.	The release gate for prompts/models — 'pytest for model behavior'. Grow the suite after every incident.
GitHub Actions + branch protection	CI pipeline that runs the eval gate on every pull request; failing check blocks the merge.	Production. (Locally the same gate ran as a git pre-push hook.)
Rolling deploy	Replace instances gradually with the new version.	Stateless code changes with low behavioral risk.
Blue-green	Two full environments; switch traffic in one move; instant switch-back.	Fast, coarse rollback matters more than gradual exposure.
Canary	Send 10% → 50% → 100% of traffic to the new version, gated by automated analysis at each step.	Default for AI releases — behavioral regressions surface only under real traffic.
Shadow deployment	New version receives a mirrored COPY of live traffic; users only ever see the old version's answers.	Highest-risk upgrades (new model version): full production signal, zero user exposure. Ladder: shadow → canary → full.
Rollback discipline	Return traffic to the previous (kept-warm) version FIRST; debug after mitigation.	Every incident. Rehearse it (game days) — rollback is seconds, debugging is hours.

4 · Observability & Governance (S4, S5, S10)

Tool	Simple definition	Use it when
Prometheus	Pull-based metrics database: scrapes /metrics text endpoints on an interval; you query with PromQL.	Numbers over time: request rate, latency histograms, error counts. The infrastructure eye.
Grafana	Dashboards and alert rules over Prometheus (and other sources); dashboards provisioned as code.	Making metrics visible and alertable. Alert on symptoms users feel (p95, error rate) — not on everything.
Evidently	Statistical drift detection: compares a baseline traffic window against current, per-feature tests + verdict.	Population-level 'did the world change?' — scheduled checks on logged features. The crowd-watcher.
Langfuse	LLM observability: one trace per request with prompts, outputs, timings, token usage and cost; self-hostable.	Interaction-level 'what happened in THIS request?' — debugging, version comparison, cost per query. The conversation-watcher.
Audit log	Append-only record per model interaction: timestamp, request id, model version, redacted prompt/response, latency.	Regulated industries — proving what the model said, when, and which version said it.
PII redaction	Removing emails/phones/IDs before storage. Lab: regex. Production: NER engines (Microsoft Presidio) + regex, measured by recall.	At LOG time when raw text must never be stored; at QUERY time when recoverability is required and storage is locked down.

Key metric vocabulary

Term	Plain definition
p95 latency	Time within which 95 of 100 requests complete. Judge services by this, not the average — averages hide the slowest users' pain.
Throughput / tokens/sec	Requests (or generated tokens) per second across the whole service. The capacity — and therefore the cost — metric.
Cold start	Launch → first useful reply; dominated by model load. The structural price of scale-to-zero.
SLI / SLO / SLA	SLI = the measured indicator; SLO = your internal target ('p95 < 4s, 99.9% of 30 days'); SLA = the contractual promise with penalties.
Error budget	The allowed failure amount implied by the SLO (99.9% ≈ 43 min/month). Budget left = ship; budget spent = stop and stabilise.
Drift (data / prediction / concept)	Inputs change / outputs change / the input→output relationship itself changes — while your code stays identical.
Trace / span / generation	One request's end-to-end record / one step inside it / the LLM-call step specifically (with tokens and cost).
Eval / LLM-as-judge	An automated behavioral test. LLM-as-judge = a strong model grades outputs against a rubric — sample-based, audited against humans.

5 · The Application Layer (S6–S10 — the ClaimAssist stack)

Tool / pattern	Simple definition	Use it when
FastAPI + Pydantic	The product API; Pydantic validates inputs and forces STRUCTURED outputs (typed JSON, not free text).	Every AI backend. Structured outputs are what make LLM responses programmable.
SSE streaming	Server-Sent Events: the answer flows token-by-token over one HTTP response.	Chat UX — time-to-first-token is the perceived speed. Pair with retries + idempotency keys on POST.
Streamlit	Python-only UI framework: a working chat app in ~50 lines.	Internal tools, analytics teams, demos — shipped in hours.
Next.js + Vercel AI SDK	Production web UI; useChat + streamText handle the streaming plumbing.	Customer-facing products: real design control, auth, deployment story.
OpenAI protocol	The de-facto standard chat API shape (/v1/chat/completions).	Speak it everywhere — then local vLLM, cloud providers and proxies are all interchangeable.
LiteLLM	A proxy/SDK that speaks OpenAI in front of 100+ providers: routing, fallbacks, retries, cost-based selection, budgets.	Never hard-code one vendor. Provider outage/quota/cost problems become config, not code.
ChromaDB (pgvector/FAISS)	Vector store: stores chunk embeddings, returns nearest chunks for a query.	RAG retrieval. Chroma = simplest local; pgvector = you already run Postgres; FAISS = library, not a server.
RAG + citations + confidence	Retrieve relevant clauses → ground the prompt → answer ONLY from context → cite sources → surface confidence; refuse when not found.	Any answer that must be verifiable. 'Not in the documents' is a feature, not a failure.
MCP	Model Context Protocol — a standard for exposing tools/resources to AI apps (the 'USB-C of tools'); solves M×N bespoke integrations.	Giving the app hands: lookup, search, draft. Split read vs write tools; write actions need human confirmation.
Prompt registry + A/B	Prompts versioned like code (version/owner/changelog); A/B two versions in production and compare traces.	Any prompt change — 'edited live in production' is the pitfall.
Guardrails (boundary)	Input checks (injection blocklist, topic allowlist) + output checks (PII redaction, refusal templates) around the LLM call.	Every user-facing AI product. Frameworks: Guardrails AI, NeMo Guardrails; the pattern matters more than the library.

6 · The Wider Ecosystem — how course tools relate to names you'll hear

The course deliberately used **direct building blocks** (openai SDK + FastAPI + LiteLLM + Chroma) so every layer stays visible. The frameworks below sit ON TOP of those blocks — same concepts, more abstraction.

Name	What it is	Relation to what you used
LangChain	Application framework: pre-built chains, prompt templates, retrievers, memory, and hundreds of integrations.	A framework layer over what you hand-wrote in S6–S8 (LLM call + retrieval + prompt assembly). Faster start, less visibility. Knowing the raw layers makes you a better LangChain user — you can debug beneath it.
LangGraph	LangChain's engine for STATEFUL agent workflows: nodes and edges (graphs), branching, loops, human-in-the-loop checkpoints.	A structured, production-oriented version of the agent loop you wrote in S9 (loop.py + max-iterations + confirmation). Choose it when agent flows get complex/multi-step.
LlamaIndex	Framework specialised in RAG: ingestion, indices, query engines over your data.	The framework version of your S8 rag/ingest.py + grounded prompt. Strong for complex document pipelines; same chunk→embed→retrieve→ground idea.
LangSmith	LangChain's commercial observability/eval platform: traces, datasets, eval runs.	Same category as Langfuse (S5/S10). Langfuse = open-source, self-hostable; LangSmith = managed, deepest LangChain integration.
Ollama	One-command local model runner for laptops (Mac/Windows/Linux), OpenAI-compatible API, no GPU required.	The developer-laptop cousin of vLLM: Ollama optimises convenience for ONE user; vLLM optimises throughput for MANY concurrent users on server GPUs.
Hugging Face transformers	The model library itself — pipeline(), tokenizers, weights hub.	You used it inside every server (S1). Serving stacks wrap it (or replace it with optimised engines) for production traffic.
CrewAI / AutoGen	Multi-agent frameworks: several role-playing agents collaborating on a task.	Beyond this course's single-agent scope (S9). Master one agent + tools + safety first; multi-agent multiplies both power and failure modes.
Ragas / DeepEval	Eval libraries: Ragas = RAG-specific metrics (faithfulness, context precision); DeepEval = pytest-style LLM tests.	Same job as your Promptfoo gates (S3), different ergonomics. The discipline (gate releases on evals) transfers unchanged.
Guardrails AI / NeMo Guardrails	Validation frameworks: schema-checked outputs (Guardrails AI) and dialog-flow rails (NeMo).	Production frameworks for the boundary checks you hand-wrote in S10 (guardrails.py).
Kubeflow / MLflow / W&B;	ML platform tooling: pipelines, experiment tracking, model registry.	The TRAINING-side neighbours of this course's serving-side stack; MLflow's registry pairs naturally with your S5 upgrade triggers.

7 · Decision One-Liners

If you need to...	Reach for
Serve an LLM to many concurrent users	vLLM (or SGLang for shared prefixes / strict JSON) behind a load balancer
Ship a quick internal demo today	FastAPI + Streamlit; upgrade when traffic demands respect
Get from model to Docker image with least friction	BentoML (build + containerize)
Deploy a container without a platform team	Managed runtime (Cloud Run / App Runner / Container Apps)
Stop a bad prompt from reaching production	Promptfoo eval gate in CI (branch protection) — evals, not code review, catch prompt regressions
Release a new model version safely	Shadow (mirror traffic, compare traces) → canary with automated analysis → 100%
Know the service is healthy right now	Prometheus + Grafana: p95, error rate, alert on SLO symptoms
Know the answers are still good	Evals on a schedule + Evidently drift on logged features
Debug one bad conversation / compare versions / see cost	Langfuse traces (tag by model/prompt version)
Avoid vendor lock-in across LLM providers	Speak the OpenAI protocol everywhere; route through LiteLLM with fallbacks
Make answers verifiable	RAG with clause-level citations + confidence + honest 'not found'
Give the app abilities (lookup, search, draft)	MCP tools + an agent loop with max-iterations; write actions gated by human confirmation
Change a prompt in production	Version it in the registry, A/B it, compare traces, then promote — never edit live
Protect users and the company	Guardrails at the boundary (input + output) + PII redaction + audit log

8 · Package & Setup Quick Reference

Layer	Install	Notes
Python env	python3 -m venv .venv && source .venv/bin/activate; pip install ipykernel	Never create venvs on network drives; kernels: python -m ipykernel install --user --name=labenv
Serving	pip install fastapi uvicorn transformers torch bentoml pip install vllm	CPU-only torch saves ~2 GB: --index-url https://download.pytorch.org/whl/cpu · vLLM needs Linux + NVIDIA
Containers	apt install docker.io docker-compose-v2 (check docker compose version FIRST — often pre-installed)	Mac/Win: Docker Desktop, 8 GB+ memory allocated. Build needs the trailing '.' — docker build -t name .
Evals / CI	Node 18+ (apt nodejs npm / brew install node); npx promptfoo@latest	Hooks must be executable: chmod +x .git/hooks/pre-push — git silently skips otherwise
Monitoring	Images: prom/prometheus, grafana/grafana; pip install prometheus-client "evidently==0.4.40"	Mount config DIRECTORIES, not single files (stale-inode issue on NFS)
Tracing	pip install langfuse; self-host: langfuse/langfuse:2 + postgres:15	Pass start/end times + usage tokens or the UI shows 0s and \$0
App layer	pip install chromadb mcp httpx streamlit; web: npm install (next@15, ai, @ai-sdk/openai)	Local model speaks /v1/chat/completions — one .env line switches local ↔ cloud ↔ LiteLLM